RedKross Research Foundation

SOC Operation and Threat Detection

PROJECT TECHNICAL REPORT

Incident Communication Generator

Python-Based Automated Incident Communication System

Organisation	RedKross Research Foundation
Department	SOC Operation and Threat Detection
Task ID	2438
Prepared by	Deepika S
Department / Course	B.E CSE(CYBERSECURITY)
Submission date	10-11-2026

The incident scenario used in this report (RK-INC-TRAIN-204, Northstar Community Services) is fictional.

Table of Contents

Chapter 1 – Introduction..... 9
1.1 Background..... 9
1.2 Problem Statement..... 9
1.3 Project Motivation..... 10
1.4 Project Objectives..... 10
1.5 Scope..... 11
1.6 Intended Users..... 11
Chapter 2 – System Requirements and Analysis..... 12
2.1 Functional Requirements..... 12
2.2 Non-Functional Requirements..... 12
2.3 Hardware and Software Requirements..... 13
2.4 Use Case Analysis..... 13
2.5 Input and Output Analysis..... 14
Chapter 3 – System Architecture and Design..... 15
3.1 Architecture Overview..... 15
3.2 System Architecture Diagram..... 15
3.3 Module-Level Architecture..... 17
3.4 Data Flow Diagram..... 17
3.5 Processing Workflow..... 18
3.6 Data Design..... 19
Chapter 4 – Implementation and Technical Explanation..... 21
4.1 Development Environment Setup..... 21
4.2 Project Directory Structure..... 21
4.3 Incident Parser Implementation..... 22
4.4 Communication Generator Implementation..... 23
4.5 Output Validation Implementation..... 24
4.6 Main Application Execution..... 25
4.7 Execution Command..... 25
4.8 Output File Structure..... 26
Chapter 5 – Generated Communication Reports..... 27
5.1 Technical Incident Update..... 27
5.2 Management Incident Update..... 28
5.3 Staff Security Notice..... 29
5.4 Audience Comparison..... 29
5.5 Generated Output Analysis..... 29
Chapter 6 – Security Controls and Reliability..... 31
Chapter 7 – Testing and Results..... 33

7.1 Testing Strategy..... 33
7.2 Test Environment..... 33
7.3 Test Categories..... 34
7.4 Test Execution Command..... 34
7.5 Test Results..... 34
7.6 Individual Test Evidence..... 35
7.7 Test Analysis..... 37
Chapter 8 – Screenshot Evidence and Implementation Verification..... 38
8.1 Evidence Map..... 38
8.2 Insertion and Formatting Guidance..... 38
8.3 Implementation Verification Checklist..... 38
Chapter 9 – Limitations and Future Enhancements..... 40
9.1 Current Limitations..... 40
9.2 Future Enhancements..... 40
Chapter 10 – Conclusion..... 41
Appendix A – Project Directory Structure..... 42
Appendix B – Important Execution Commands..... 43
Appendix C – Evidence Index..... 44
Appendix D – Sample Incident Data..... 45
Appendix E – Test Summary..... 46

List of Figures

Figure 1 – Use Case Diagram....14
Figure 2 – Overall System Architecture....16
Figure 3 – Level 0 Data Flow Diagram....17
Figure 4 – Level 1 Data Flow Diagram....18
Figure 5 – Application Workflow Diagram....19
Figure 6 – Incident Input Validation Flow....23
Figure 7 – Audience-Specific Communication Generation....24
Figure 8 – Three-Audience Output Comparison Diagram....30
Figure 9 – Final Automated Test Summary....35
Figure 10 – Successful execution of incident parser unit tests....36
Figure 11 – Communication generator unit test results....36
Figure 12 – Output validation unit test results....36
Figure 13 – Generated output file verification results....37
Figure 14 – Generated technical incident update....28
Figure 15 – Generated management incident update....28
Figure 16 – Generated staff security notification....29
Figure 17 – Final automated test execution summary showing 28 passed and 0 failed....35
Figure 18 – Security Controls Mapping Diagram....32
Figure 19 – Testing and Verification Flow Diagram....33

List of Tables

Table 1 – Functional requirements.....12
Table 2 – Verified software environment.....13
Table 3 – Input and output files.....14
Table 4 – Module responsibilities.....17
Table 5 – Incident JSON fields.....20
Table 6 – Fictional scenario: confirmed and not-yet-confirmed information.....27
Table 7 – Comparison of the three audience-specific reports.....29
Table 8 – Risk-to-control mapping.....32
Table 9 – Automated test summary by module.....34
Table 10 – Final automated test results.....34
Table 11 – Screenshot evidence map.....38
Table 12 – Execution commands.....43
Table 13 – Evidence index.....44
Table 14 – Test summary (verified results).....46

Chapter 1 – Introduction

1.1 Background

Cybersecurity incident response is the organised process of detecting, analysing, containing and recovering from events that threaten the confidentiality, integrity or availability of information systems. A Security Operations Centre (SOC) carries out much of this work: analysts monitor telemetry, triage alerts, investigate suspicious activity and coordinate the response. Technical investigation, however, is only half of the task. Every incident also generates a stream of communication, and the people who need to be told what is happening differ widely in their knowledge, their responsibilities and the decisions they must make.

Incident communication is therefore an operational control in its own right. Technical responders need precise detail so that they can continue the investigation without duplicating effort. Service owners need a clear picture of the current situation and its business impact so that they can decide whether to escalate, allocate resources or inform others. Ordinary staff need short, plain guidance on what they should do and what they should report. When these updates are late, inconsistent or written for the wrong audience, the result is confusion, wasted effort and, in the worst case, harmful decisions taken on the basis of incorrect information.

Four qualities define good incident communication. It must be **timely**, so that readers can act while the information is still relevant; **accurate**, so that every statement is supported by evidence; **consistent**, so that different audiences are not given contradictory accounts; and **audience-appropriate**, so that each reader receives the level of detail and the guidance that suits their role. Achieving all four while an incident is still unfolding is difficult, because analysts are working under pressure and the facts are changing.

1.2 Problem Statement

When incident updates are prepared manually, the same facts are rewritten several times for several audiences. This repetition introduces predictable problems:

- **Inconsistent information across reports.** A detail corrected in one update may remain unchanged in another, so that responders, managers and staff are working from different versions of the incident.

- **Excessive technical detail for non-technical audiences.** Staff and service owners may receive logs-level terminology that they cannot interpret, which obscures the message they actually need.
- **Missing important incident information.** Under time pressure, the incident identifier, status or a required caveat may simply be left out.
- **Confusion between confirmed facts and assumptions.** Hypotheses that are worded confidently can be read as findings, and findings can be diluted by hedging.
- **Missing next-update schedules or contact routes.** Readers are left without knowing when to expect further information or whom to contact.
- **Unsupported claims and premature attribution.** An update may name a person as responsible, or promise a recovery time, before the evidence supports either statement.

1.3 Project Motivation

A structured communication-generation approach addresses these problems at their source. If the incident facts are captured once, in a defined structure, every audience-specific report can be produced from the same record. Consistency then becomes a property of the process rather than something each author must remember to maintain. A schema can reject incomplete input before any report is written, and the generated reports can be checked for the elements that must never be missing, such as the incident identifier, the next update time and the contact route.

The Incident Communication Generator was developed under Task ID 2438 for the SOC Operation and Threat Detection domain of the RedKross Research Foundation to demonstrate this approach in a small, reviewable, fully tested Python application. The emphasis is on discipline rather than sophistication: separating what is confirmed from what is not, refusing to speculate about responsibility, and refusing to promise recovery times that have not been validated.

1.4 Project Objectives

The project was built to meet the following measurable objectives:

1. Load structured incident information from a JSON file.
2. Validate the incident input against a JSON Schema (Draft 2020-12) before any processing.

3. Generate three audience-specific communications: a technical update, a management update and a staff security notice.
4. Preserve confirmed facts and not-yet-confirmed information as separate categories in every report.
5. Require the operator to provide the next update time (UTC) and an approved contact route, rather than inventing them.
6. Validate each generated report for required elements.
7. Save the reports as Markdown files in designated output directories.
8. Verify the behaviour of the application through automated tests (28 tests, all of which pass).

1.5 Scope

Included in the current implementation. Loading structured incident data from JSON; schema validation of that data; generation of three predefined audience-specific reports; operator-supplied next update time and approved contact route; structural validation of generated output; writing of Markdown files; and an automated test suite built on Python's standard `unittest` framework.

Excluded from the current implementation. Live integration with a SIEM or SOAR platform; automatic retrieval of the next update time or the contact route; semantic verification of incident statements; authentication, encryption, access control, role-based permissions or audit logging; configurable templates; and any graphical or web interface. These items are discussed only as future enhancements in Chapter 9 and are not features of the system.

1.6 Intended Users

- **Technical responders** – analysts and engineers who investigate the incident and need operational detail, evidence and recommended technical actions.
- **Service owners** – managers accountable for the affected service, who need the current situation, the known business impact and the open questions.
- **Staff members** – the wider user population, who need short and clear security guidance and a reporting route.

- **SOC communication coordinators** – the operators who run the application, supply the next update time and approved contact route, and review every generated report before it is distributed.

Chapter 2 – System Requirements and Analysis

2.1 Functional Requirements

The functional requirements below describe what the system must do. Each requirement is traceable to a project objective and is exercised by at least one module of the automated test suite described in Chapter 7.

Table 1 – Functional requirements

ID	Requirement	Description
FR-01	Load incident data	Read structured incident information from a JSON file.
FR-02	Validate input	Validate the incident data against a JSON Schema (Draft 2020-12) and report violations.
FR-03	Next update time	Accept the next update time (UTC) from the operator; it is not generated or retrieved automatically.
FR-04	Contact route	Accept an approved contact route from the operator; it is not generated or retrieved automatically.
FR-05	Technical update	Generate a Technical Incident Update for technical responders.
FR-06	Management update	Generate a Management Incident Update for the service owner.
FR-07	Staff notice	Generate a Staff Security Notice for staff members.
FR-08	Output validation	Check each generated report for required elements before it is saved.
FR-09	Save outputs	Write each report as a Markdown file in its designated output directory.
FR-10	Separate evidence	Keep confirmed facts and not-yet-confirmed information distinct in each report.

2.2 Non-Functional Requirements

Reliability. The application must behave identically for identical input. Schema validation before processing prevents malformed data from producing partially correct reports, and the automated tests act as a regression safeguard.

Maintainability. Responsibilities are divided between a parser, a generator, a validator and a main entry point, so that a change to report wording or to validation rules affects a small and predictable part of the code.

Readability. Reports are written in Markdown, a plain-text format that is readable in any editor and renders cleanly where Markdown is supported. The generated text is deliberately short and plainly worded.

Data consistency. All three reports are produced from one validated dataset, so that incident identifier, status, confirmed facts and not-yet-confirmed information are identical wherever they appear.

Input validation and error handling. Missing files, malformed JSON and schema violations are detected and reported rather than ignored. Missing operator values are rejected rather than replaced with defaults.

Usability. The operator runs a single command and answers two prompts. The generated files are placed in clearly named directories.

2.3 Hardware and Software Requirements

Only software that has been verified for this project is listed. No hardware specification is stated, because none was measured or specified for the project.

Table 2 – Verified software environment

Component	Detail	Role
Operating system	Windows 11	Development and execution platform
Language	Python 3.12	Implementation language
Shell	PowerShell	Environment activation and command execution
Dependency	jsonschema 4.26.0	JSON Schema validation of incident input
Test framework	Python unittest	Automated test execution
Output format	Markdown	Format of the generated reports

A code editor such as Visual Studio Code may be used for development, but the project information supplied for this report does not confirm which editor was used, so none is claimed.

2.4 Use Case Analysis

The application has a single actor, the **operator** – typically the SOC communication coordinator. The operator starts the application, which loads the incident data and validates it against the schema. The operator then enters the next update time (UTC) and the approved contact route. The application generates the three audience-specific reports, validates them and saves them. The operator finally reviews the generated files before deciding whether to distribute them. Figure 1 shows these interactions.

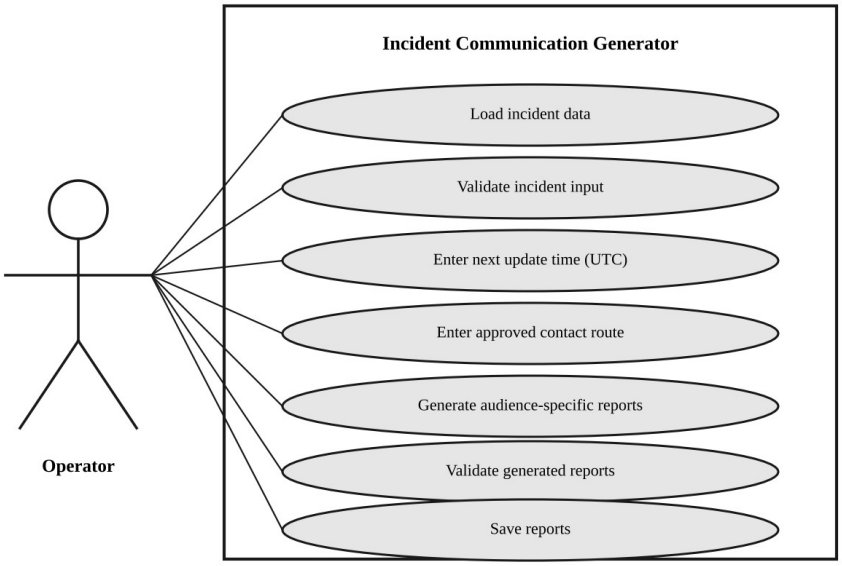


Figure 1 – Use Case Diagram

2.5 Input and Output Analysis

The system reads one structured incident file and one JSON Schema, and writes three Markdown reports. The roles in Table 3 are derived from the verified project structure; the report author should confirm them against the final repository before submission.

Table 3 – Input and output files

File	Purpose	Role
incident_facts.json	Structured incident record (project root)	Input
data/sample_incidents.json	Sample incident data	Input / sample data
data/incident_schema.json	JSON Schema defining valid incident structure	Validation rules
status_update_template.md	Markdown template file at project root	Supporting document
outputs/technical_report/technical_update.md	Technical Incident Update	Output
outputs/management_updates/management_update.md	Management Incident Update	Output
outputs/user_notifications/staff_notification.md	Staff Security Notice	Output

The next update time and approved contact route are not stored in any of these files. They are supplied by the operator at run time and inserted into every report.

Chapter 3 – System Architecture and Design

3.1 Architecture Overview

The application follows a layered, modular design in which each stage of processing has a single, well-defined responsibility. Data enter through an input layer, pass through parsing and schema validation, are combined with the operator's inputs, and are then transformed by the communication generator into audience-specific reports. Each report is checked by an output validator and finally written to disk by a file-writing step. Because the stages are separate, each can be tested independently, which is reflected in the four test modules.

3.2 System Architecture Diagram

Figure 2 shows the complete processing pipeline, with each layer labelled. Dashed rounded boxes denote inputs, grey rectangles denote processing steps, and dark boxes denote generated outputs.

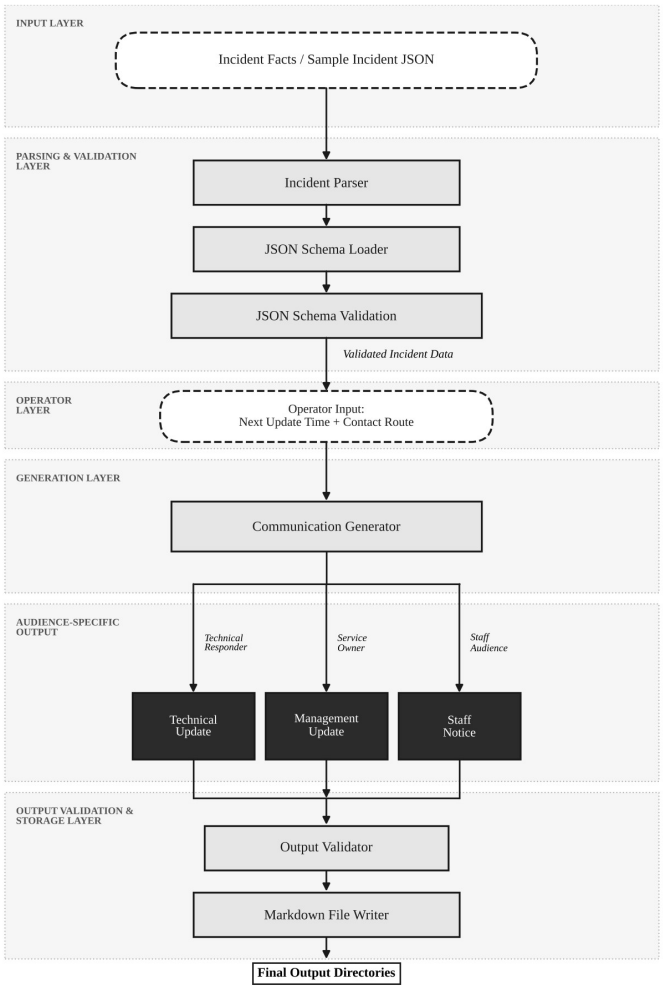


Figure 2 – Overall System Architecture

3.3 Module-Level Architecture

Table 4 – Module responsibilities

Module	Responsibility
incident_parser.py	Loads and validates structured incident data (`load_json()`, `validate_incident()`, `load_incident()`)
communication_generator.py	Creates audience-specific communication (`generate_communication()`, `technical_update()`, `management_update()`, `staff_update()`)
validator.py	Checks generated communication structure and required fields (`validate_output()`)
main.py	Coordinates application execution: loading, operator input, generation, validation and file writing
templates.py	Present in the verified directory structure. Its exact role has not been verified for this report; if the file is empty or unused by the current code, the report author must state this accurately here.

The disclosure about `templates.py` is deliberate. The directory listing confirms that the file exists, but nothing in the verified project information establishes that the generator imports from it. Describing a role that cannot be confirmed would violate the accuracy rules that govern this report.

3.4 Data Flow Diagram

Figure 3 presents the context-level (Level 0) view: the incident data source and the operator are external entities that supply data to a single process, which produces the three communications.

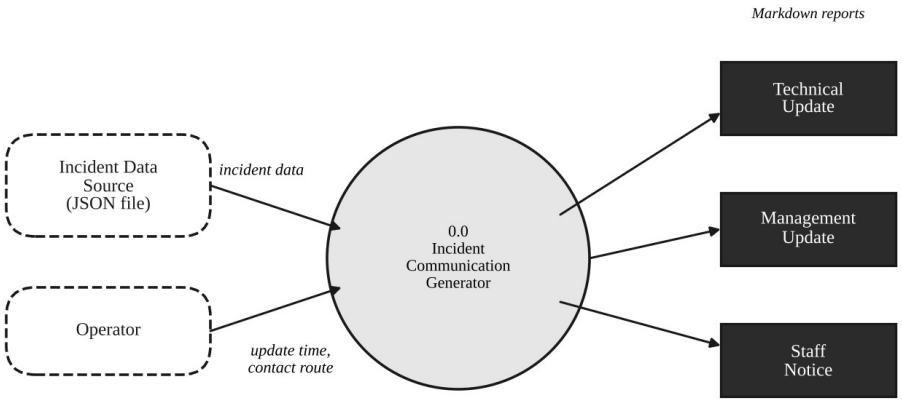


Figure 3 – Level 0 Data Flow Diagram

Figure 4 decomposes that process into seven sub-processes, from loading the incident data to storing validated reports.

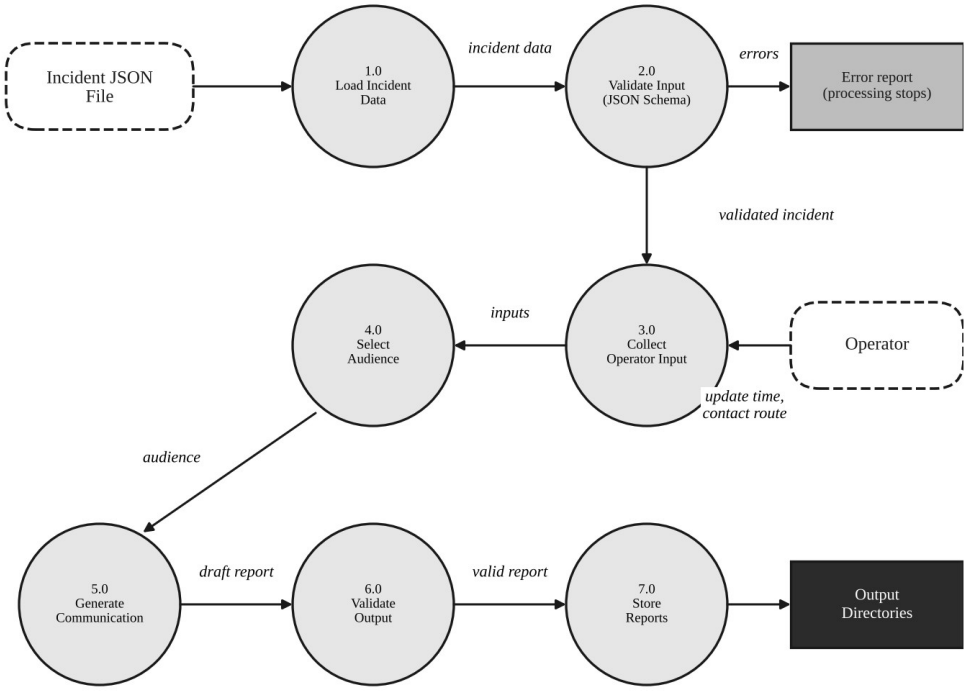


Figure 4 – Level 1 Data Flow Diagram

3.5 Processing Workflow

Figure 5 shows the numbered application workflow with its five decision points. Each decision has an error path: when a check fails, the failure is reported to the operator and the run does not proceed to produce reports. The diagram represents the designed control flow of the pipeline; the precise wording of messages is determined by the implementation.

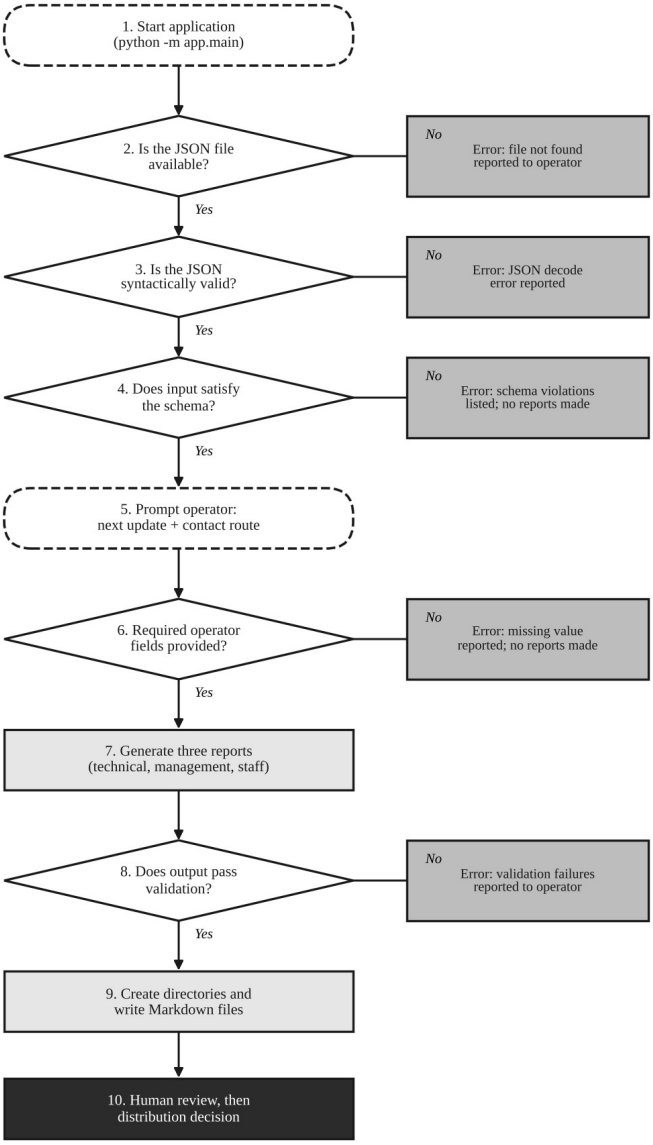


Figure 5 – Application Workflow Diagram

3.6 Data Design

The incident record is a JSON object with the following fields:

Table 5 – Incident JSON fields

Field	Description
incident_id	Unique incident identifier (RK-INC-TRAIN-204 in the scenario)
fictional_organisation	The fictional organisation affected (Northstar Community Services)
status	Current incident status (Investigating)
confirmed_facts	Statements supported by evidence
not_yet_confirmed	Statements that have not been verified
audiences	The audiences for which communication is produced
communication_rules	Rules that govern what the reports may and may not say

The **JSON Schema** is a formal description of what a valid incident record looks like. It is the mechanism by which the parser decides whether the input is acceptable: a record that violates the schema is rejected with a list of the violations, and no report is generated from it. Using a schema rather than ad hoc checks keeps the validation rules in one declarative file (`data/incident_schema.json`) that can be reviewed independently of the Python code. The project uses the Draft 2020-12 dialect of JSON Schema through the jsonschema 4.26.0 library.

Chapter 4 – Implementation and Technical Explanation

This chapter explains the implementation module by module. It describes what each component is responsible for and how it behaves, using the function names and file structure verified for the project. Source code is not reproduced beyond the verified function signature and commands, so that nothing in this report depends on unverified code.

4.1 Development Environment Setup

The project is developed in an isolated Python virtual environment so that its dependency (jsonschema 4.26.0) does not interfere with other Python projects on the same machine. The following verified PowerShell commands prepare the environment:

```
python -m venv .venv
Set-ExecutionPolicy -Scope Process -ExecutionPolicy Bypass -Force
.\.venv\Scripts\Activate.ps1
pip install -r requirements.txt
```

- `python -m venv .venv` creates a virtual environment in the `.venv` directory.
- `Set-ExecutionPolicy -Scope Process -ExecutionPolicy Bypass -Force` relaxes the PowerShell script-execution policy for the current terminal session only, so that the activation script is allowed to run. Because the scope is `Process`, the machine-wide policy is not changed.
- `.\.venv\Scripts\Activate.ps1` activates the environment, so that `python` and `pip` refer to the isolated copy.
- `pip install -r requirements.txt` installs the dependencies listed in `requirements.txt`, including jsonschema 4.26.0.

4.2 Project Directory Structure

The clean project tree is shown below. Virtual-environment files, Python cache files and the temporary `project_structure.txt` are excluded from this tree.

```
Incident_Communication_Generator/
|
|-- app/
|   |-- __init__.py
|   |-- main.py
|   |-- incident_parser.py
|   |-- communication_generator.py
|   |-- templates.py
|   `-- validator.py
|
|-- data/
|   |-- sample_incidents.json
|   `-- incident_schema.json
|
|-- tests/
|   |-- test_parser.py
|   |-- test_generator.py
|   |-- test_validator.py
|   `-- test_outputs.py
|
|-- outputs/
|   |-- technical_report/technical_update.md
|   |-- management_updates/management_update.md
|   `-- user_notifications/staff_notification.md
|
|-- evidence/   (eight screenshot files, see Appendix C)
|-- docs/       (architecture.md, test_documentation.md)
|
|-- incident_facts.json
|-- status_update_template.md
|-- requirements.txt
|-- README.md
`-- .gitignore
```

The `app` package holds the application code, `data` holds the schema and sample data, `tests` holds the four test modules, `outputs` receives the generated reports, `evidence` holds the test and output screenshots, and `docs` holds supporting documentation. The full tree, including the individual evidence filenames, appears in Appendix A.

4.3 Incident Parser Implementation

`incident_parser.py` is responsible for turning a file on disk into trustworthy, validated incident data. It does this through three functions with separate responsibilities.

- **`load_json()`** reads a JSON file and decodes it. It is used for both the incident file and the schema file. A missing file and a file that is not valid JSON are distinct failure conditions, and each is reported rather than silently ignored.
- **`validate_incident()`** checks a loaded incident against the loaded schema using a Draft 2020-12 validator from the jsonschema library. If the incident violates the schema, the violations are reported so that the operator can correct the input.

- **`load_incident()`** combines the two: it loads the incident, loads the schema, validates one against the other and returns the validated incident data to the caller.

The design principle is “validate at the boundary”. No other module receives incident data that has not passed through this validation, so the generator and validator can assume that the structure is correct. Figure 6 illustrates the flow.

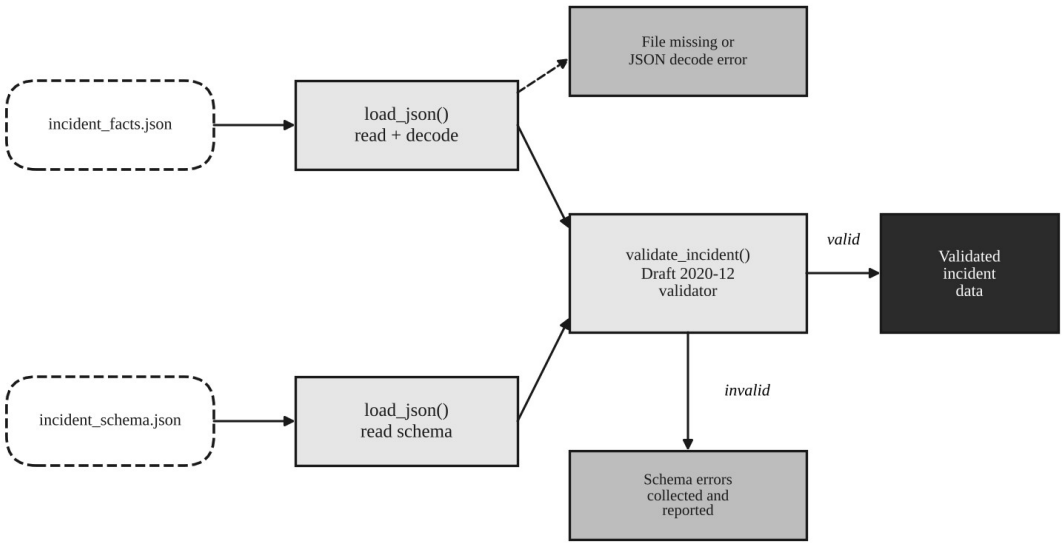


Figure 6 – Incident Input Validation Flow

4.4 Communication Generator Implementation

`communication_generator.py` contains the core transformation. Its entry point has the following verified signature:

```
generate_communication(incident, audience, next_update, contact_route)
```

The function performs the following steps:

1. **Audience validation.** The requested audience must be one of the three supported audiences; an unsupported value is rejected.
2. **Required operator input checks.** The `next_update` and `contact_route` values must be provided. The function does not invent or default them.
3. **Formatting confirmed facts.** The incident’s confirmed facts are formatted for inclusion under a clearly marked confirmed-information heading.

4. **Formatting unconfirmed information.** The not-yet-confirmed items are formatted separately and marked as unconfirmed, so that a reader cannot mistake them for findings.
5. **UTC timestamp generation.** A timestamp expressed in UTC is generated so that the report states explicitly when it was produced.
6. **Audience-specific function selection.** According to the audience, the call is dispatched to one of the three report-generation functions.

The three report-generation functions are:

- **`technical_update()`** – produces the Technical Incident Update for responders, with operational detail, investigation requirements and recommended technical actions.
- **`management_update()`** – produces the Management Incident Update for the service owner, centred on the current situation and the known business impact.
- **`staff_update()`** – produces the Staff Security Notice, written in plain language with precautions and a reporting route.

All three receive the same incident data, formatted fact lists, timestamp, next update time and contact route. The differences between the reports are therefore differences of emphasis and wording, not of fact. Figure 7 shows this branching.

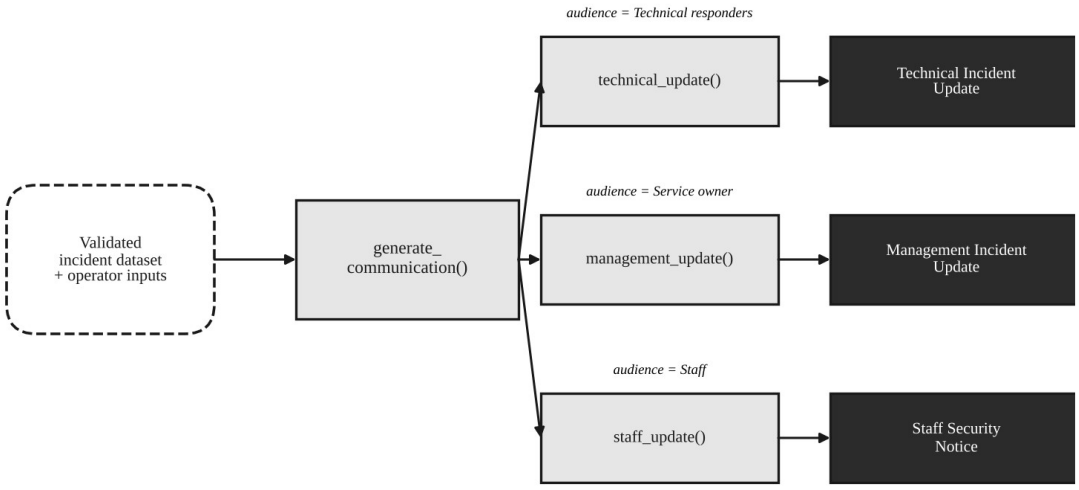


Figure 7 – Audience-Specific Communication Generation

4.5 Output Validation Implementation

`validator.py` provides the `validate_output()` function, which examines a generated report before it is saved. It checks for the presence of:

- the incident ID;
- the next update field;
- the contact field;
- the confirmed-information headings; and
- the indicators that mark unconfirmed information.

Instead of stopping at the first problem, the function collects every failed check, so that the operator sees the complete list of defects. It then reports whether the output is valid or invalid.

Important limitation. The current validator performs structural and required-indicator checks only. It confirms that the required elements are present in the report. It does **not** independently establish the truth of any incident statement, and it does not perform full semantic verification. A report can pass validation and still contain an inaccurate statement if the incident data supplied by the analyst were inaccurate, which is why human review remains essential.

4.6 Main Application Execution

`main.py` coordinates the run. Its responsibilities, in order, are:

1. load the validated incident through the parser;
2. display the incident metadata (such as the incident ID and status) to the operator;
3. prompt the operator for the next update time and the approved contact route;
4. iterate over the three audiences;
5. generate each report with `generate_communication()`;
6. validate each report with `validate_output()`;
7. create the destination directories if they do not already exist;
8. write each report to its Markdown file; and
9. handle errors so that a failure is reported to the operator rather than producing misleading output.

File handling uses Python's `pathlib.Path` class. A `Path` object represents a file-system location independently of the operating system's separator conventions. The `mkdir()` method creates the output directory, and `write_text()` writes the report text to the file in a single call. Together they make the file-writing step short and easy to verify.

4.7 Execution Command

The application is run as a module from the project root with the verified command:

```
python -m app.main
```

Running with `-m` ensures that the `app` package is resolved correctly. The operator is shown the incident metadata, is prompted for the next update time (UTC) and the approved contact route, and the application then writes three Markdown files, one per audience.

4.8 Output File Structure

The three generated reports are saved at these exact locations:

```
outputs/technical_report/technical_update.md
outputs/management_updates/management_update.md
outputs/user_notifications/staff_notification.md
```

Each audience has its own directory, so reports can be reviewed and distributed to the appropriate group without mixing them.

Chapter 5 – Generated Communication Reports

This chapter explains the content and purpose of each generated report using the fictional incident RK-INC-TRAIN-204 affecting Northstar Community Services (status: Investigating). The scenario is fictional and is not a real-world incident. It contains three confirmed facts – repeated failed sign-in events for one account, two unexpected MFA prompts denied by the user, and a privilege event with an unclear change-ticket reference – and three items that are **not yet confirmed**: whether credentials were stolen, whether data was accessed or copied, and whether the privilege event was unauthorised.

Why the distinction matters. In incident communication, a confirmed fact is a statement that evidence supports; an unconfirmed item is a hypothesis that is still being tested. If a hypothesis is presented as a fact, readers may take unnecessary or harmful action – for instance, announcing a data breach that did not occur, or blaming an individual who is innocent. If a confirmed fact is hedged until it sounds uncertain, readers may under-react. Keeping the two categories explicitly separate lets each reader calibrate their response to the actual state of knowledge, protects the organisation from making claims it cannot support, and preserves the credibility of later updates when the facts change.

Table 6 – Fictional scenario: confirmed and not-yet-confirmed information

Confirmed facts	Not yet confirmed
Repeated failed sign-in events were recorded for one account.	Whether credentials were stolen.
The user denied two unexpected MFA prompts.	Whether data was accessed or copied.
A privilege event has an unclear change-ticket reference.	Whether the privilege event was unauthorised.

5.1 Technical Incident Update

The Technical Incident Update is written for technical responders. It contains the incident metadata (identifier, organisation and status), the confirmed facts, the investigation requirements, the recommended technical response actions, the known impact, and the next update time and contact route.

Responders need operational detail because their task is to continue the investigation: they must know exactly what has been observed so that they can decide which logs to examine, which accounts to review and which hypotheses to test. The recommended technical actions

in the report are **recommendations**; the report does not state that any of them has been carried out. The not-yet-confirmed items appear as open questions, framing the investigation rather than concluding it.



Figure 14 – Generated technical incident update

Figure 14 is the evidence for this report. It should show the audience-specific technical sections described above: the incident metadata, the separate confirmed and unconfirmed lists, the investigation and response sections, and the next update and contact fields.

5.2 Management Incident Update

The Management Incident Update is written for the service owner. It describes the current situation, separates confirmed from unconfirmed information, states the business impact that is currently known, lists recommended response actions, and gives the next update time and contact route.

Service owners make decisions about escalation, resourcing and wider notification. They need a clear and proportionate account: what is known, what is not, and what is being done. Management communication must not exaggerate. Overstating the findings can trigger unnecessary escalation, while understating them can delay necessary decisions. Where the impact is not yet established, the report says so instead of estimating it, and it does not promise a recovery time that has not been validated.



Figure 15 – Generated management incident update

Figure 15 should show how the report communicates the current situation and the business impact in non-technical terms, with uncertainty stated explicitly.

5.3 Staff Security Notice

The Staff Security Notice is written for the general user population. It tells staff what they should know, what remains unconfirmed, which precautions to take, how to report suspicious activity, and which approved contact route to use.

Staff do not need technical detail; they need to know what to do and whom to contact. The notice therefore uses plain language and short statements. It also avoids naming any person as responsible and avoids speculation, because an unsupported statement circulated to the whole organisation is difficult to retract.

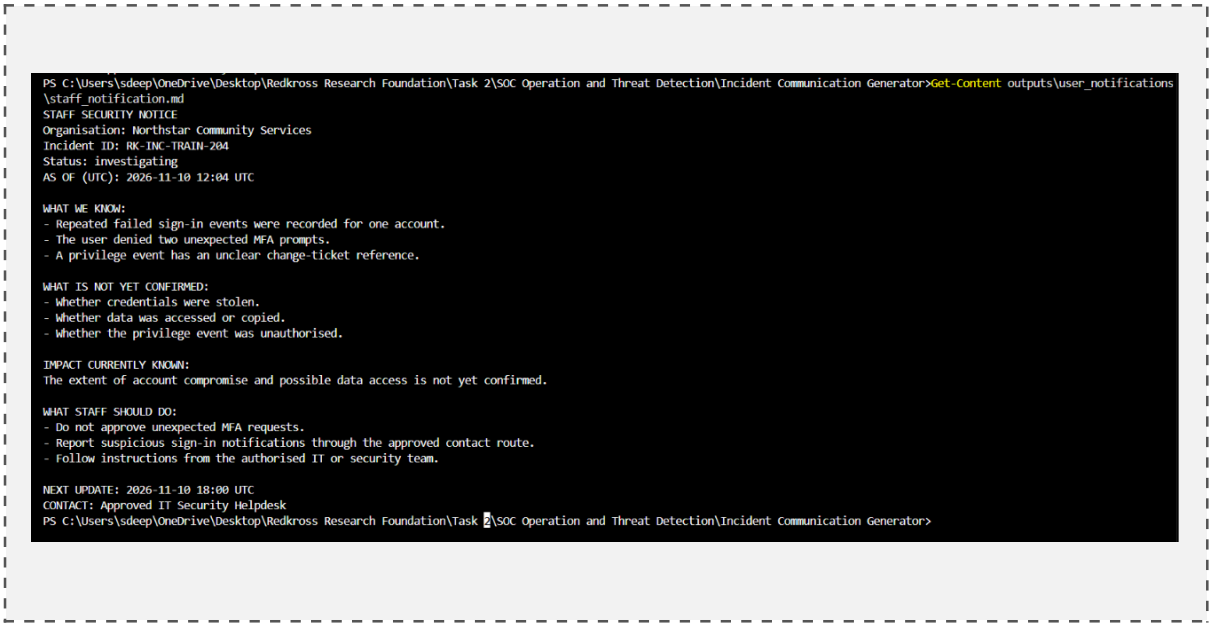


Figure 16 – Generated staff security notification

Figure 16 should show the user-facing precautions and the reporting instructions, together with the approved contact route supplied by the operator.

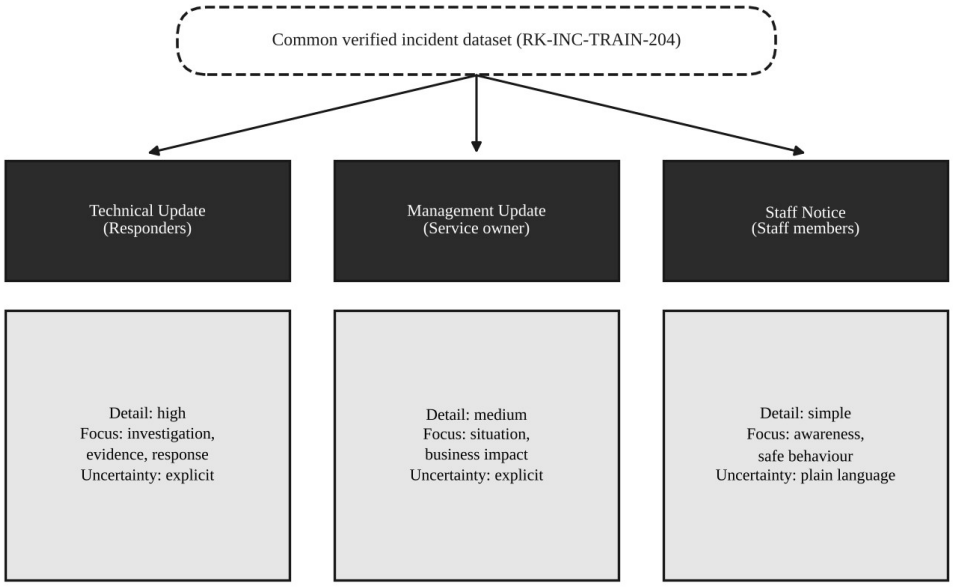
5.4 Audience Comparison

Table 7 – Comparison of the three audience-specific reports

Aspect	Technical Responders	Service Owner	Staff
Main focus	Technical investigation	Operational / business situation	User awareness
Detail level	High	Medium	Simple
Actions	Technical response	Oversight and impact assessment	Safe user behaviour
Uncertainty	Explicit	Explicit	Clear and understandable

5.5 Generated Output Analysis

A single validated incident dataset is transformed into three formats. The facts do not change between reports: the same incident identifier, the same status, the same confirmed facts and the same unconfirmed items appear in all three. What changes is the selection of material, the level of detail and the tone. This is the central design idea of the project – consistency of content with adaptation of presentation. Figure 8 summarises the relationship.



Shared in all three: incident ID, status, confirmed vs not-yet-confirmed split, next update (UTC), approved contact route

Figure 8 – Three-Audience Output Comparison Diagram

Chapter 6 – Security Controls and Reliability

The project does not implement security infrastructure such as encryption, authentication, access control, role-based permissions, audit logging or live SOC integration, and none is claimed. The controls below are the safeguards that are actually built into the communication workflow, together with the reason each one matters.

1. **Structured input validation.** Incident data must be well-formed JSON. This stops malformed or truncated records from silently producing unreliable communication.
2. **JSON Schema enforcement.** The schema defines the required structure, so missing or mis-typed fields are caught before any report is written.
3. **Required operator inputs.** The next update time and approved contact route must be supplied by the operator. This prevents the system from inventing commitments or directing readers to an unapproved contact.
4. **Separation of confirmed and unconfirmed information.** Hypotheses cannot be read as findings, which limits the risk of unwarranted escalation or false reassurance.
5. **No unsupported personal attribution.** The communication rules forbid naming a person as responsible without evidence, protecting individuals from premature blame and the organisation from defamation and morale damage.
6. **No unverified recovery-time promises.** Reports do not commit to a recovery time until it has been validated, avoiding commitments that may later be broken.
7. **Audience-specific communication.** Each audience receives the level of detail it needs and no more, reducing confusion and unnecessary exposure of technical detail.
8. **Output validation.** Required elements – incident ID, next update, contact, confirmed headings and unconfirmed indicators – are checked before a report is saved.
9. **Explicit UTC timestamp.** Reports carry a timestamp in UTC, removing time-zone ambiguity in a distributed response.
10. **Human review before distribution.** The generated reports are drafts for an accountable person to review. This is a procedural control rather than a software feature, and it compensates for the validator’s inability to verify truth.

Table 8 – Risk-to-control mapping

Risk	Implemented control
Malformed or incomplete incident input	JSON parsing and JSON Schema validation (incident_parser.py)
Missing update time or contact route	Required operator inputs (generator and main.py)
Hypotheses presented as facts	Separate confirmed and not-yet-confirmed sections; output validation of headings and indicators
Blame assigned without evidence	Communication rule: no unsupported attribution
Premature recovery-time commitments	Communication rule: no recovery time until validated
Reports missing key fields	validate_output() checks required elements
Wrong level of detail for audience	Three audience-specific generation functions
Unreviewed release of a report	Human review before distribution (procedural)

Figure 18 presents the same mapping graphically.

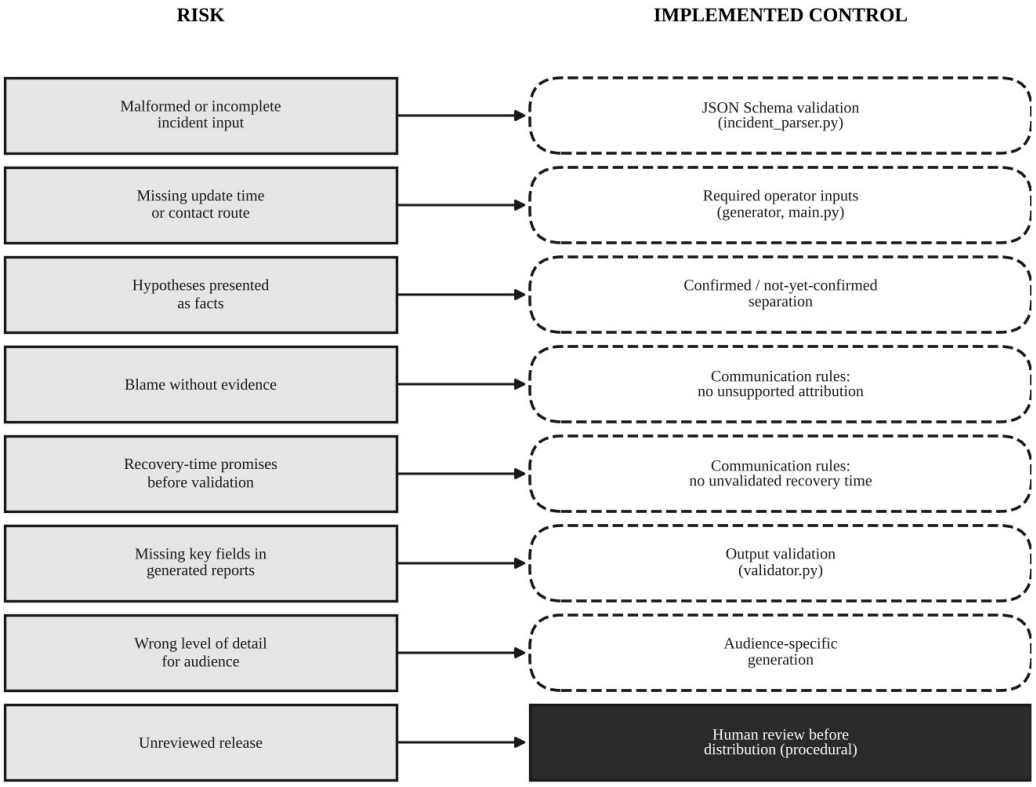


Figure 18 – Security Controls Mapping Diagram

Chapter 7 – Testing and Results

7.1 Testing Strategy

Testing combines several complementary approaches, all executed through Python's `unittest` framework:

- **Unit testing** exercises individual functions in isolation.
- **Functional testing** checks that the components together produce the intended reports.
- **Input validation testing** confirms that incident data is accepted or rejected correctly.
- **Output validation testing** confirms that generated reports are checked for required elements.
- **File-generation testing** confirms that the report files are created as expected.

Figure 19 shows how the four test modules fit together under one command.

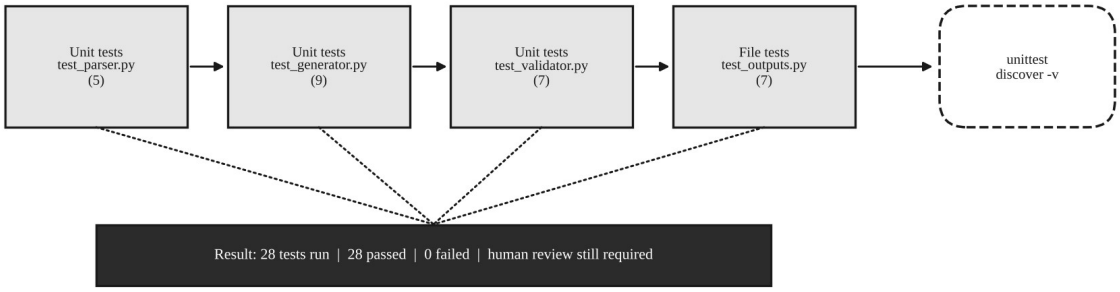


Figure 19 – Testing and Verification Flow Diagram

7.2 Test Environment

- Python 3.12
- jsonschema 4.26.0
- Python unittest

7.3 Test Categories

Table 9 – Automated test summary by module

Test Module	Tests	Result
test_parser.py	5	Passed
test_generator.py	9	Passed
test_validator.py	7	Passed
test_outputs.py	7	Passed
Total	28	28 Passed, 0 Failed

7.4 Test Execution Command

```
python -m unittest discover -s tests -v
```

The command starts `unittest` in discovery mode (`discover`), searches the `tests` directory (`-s tests`) for test modules whose names begin with `test`, and runs every test found. The verbose flag (`-v`) prints each test's name and result on its own line instead of a row of dots. Verbose mode is useful because a reviewer can see exactly which tests ran and which, if any, failed, rather than relying on a bare total.

7.5 Test Results

The recorded outcome of the test run is: **total tests: 28; passed: 28; failed: 0.**

Table 10 – Final automated test results

Metric	Value
Total tests	28
Passed	28
Failed	0

Figure 9 shows the distribution of the 28 passing tests across the four modules. All values are the verified counts.

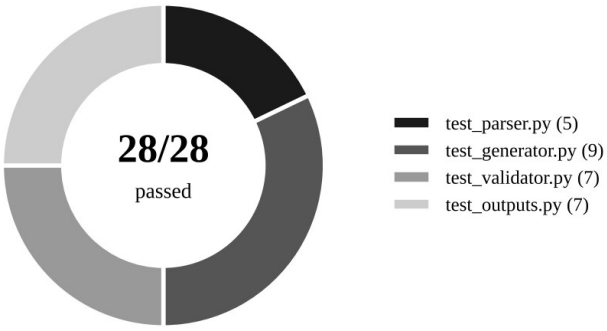


Figure 9 – Final Automated Test Summary



Figure 17 – Final automated test execution summary showing 28 passed and 0 failed

Figure 17 is the overall test-suite evidence. It should show the complete run reporting 28 tests passed and none failed.

7.6 Individual Test Evidence

Individual test names and outputs are not listed here because they have not been verified for this report; the screenshots below are the authoritative record of what each module executed.

7.6.1 Parser tests

The five parser tests (test_parser.py) exercise the code that loads and validates incident input. Their purpose is to demonstrate that valid input is accepted and that the parser's boundary checks are enforced.

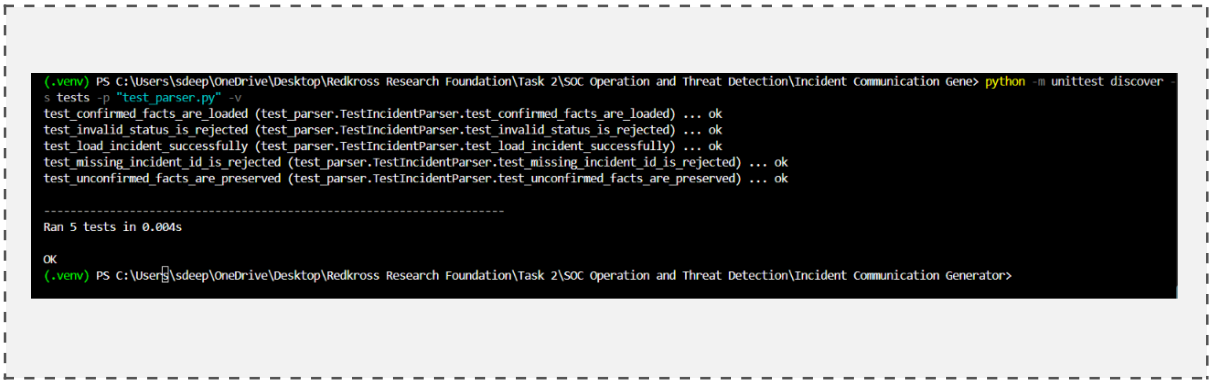


Figure 10 – Successful execution of incident parser unit tests

Figure 10 demonstrates that the parser-related tests were executed successfully.

7.6.2 Generator tests

The nine generator tests (test_generator.py) are the largest group, reflecting the generator's central role. They check that audience-specific communication is produced from the incident data and the operator's inputs.

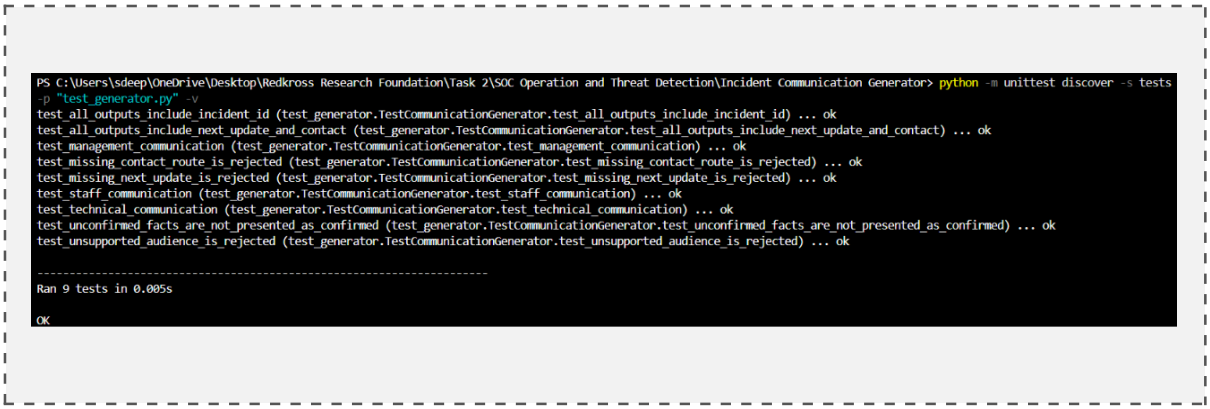


Figure 11 – Communication generator unit test results

Figure 11 shows the generator test results, all of which passed.

7.6.3 Validator tests

The seven validator tests (test_validator.py) check that output validation behaves correctly, including its detection of missing required elements.



Figure 12 – Output validation unit test results

Figure 12 shows that the output-validation tests passed.

7.6.4 Output-file tests

The seven output tests (test_outputs.py) verify file generation: that the report files are produced in the designated locations.



Figure 13 – Generated output file verification results

Figure 13 provides the evidence that the output-file tests passed.

7.7 Test Analysis

What the tests demonstrate. All 28 tests pass, which shows that the parser, generator, validator and file-writing behaviour match the behaviour the tests specify. It provides regression protection: a later change that breaks verified behaviour would be detected.

What the tests do not prove. Passing unit tests do not establish that incident information is factually true, because the system reports what the analyst supplies. They do not demonstrate semantic correctness of the generated wording, and they do not show that the application is production-ready: no performance, security-hardening or integration testing is claimed. The result should be read as evidence of correct, specified behaviour in a controlled environment, not as certification for operational use.

Chapter 8 – Screenshot Evidence and Implementation Verification

Evidence for specific claims is placed beside the section it supports rather than collected in one appendix. This chapter explains how the evidence is organised and how to complete and verify it. The screenshots are **not embedded** in this version of the report: each location contains a clearly marked placeholder naming the exact file to insert. No screenshots have been fabricated and no terminal output has been recreated.

8.1 Evidence Map

Table 11 – Screenshot evidence map

Figure	Filename	Demonstrates	Location
10	Parser_Test_Results.png	Parser tests passing	Section 7.6.1
11	Generator_Test_Results.png	Generator tests passing	Section 7.6.2
12	Validator_Test_Results.png	Validator tests passing	Section 7.6.3
13	Output_File_Test_Results.png	File-generation tests passing	Section 7.6.4
14	Technical_Update_Output.png	Generated technical update	Section 5.1
15	Management_Update_Output.png	Generated management update	Section 5.2
16	Staff_Notification_Output.png	Generated staff notification	Section 5.3
17	Final_Test_Summary.png	28 passed, 0 failed	Section 7.5

8.2 Insertion and Formatting Guidance

- Use the original screenshots from the `evidence` directory without altering their technical content.
- Centre each screenshot and set a consistent width of about 6 inches, keeping the aspect ratio (do not stretch).
- Keep the caption directly below the image and make sure the text in the image remains readable.
- Replace each placeholder box with the image; keep the caption and explanatory sentence.

8.3 Implementation Verification Checklist

- The directory structure in Appendix A matches the final repository.

- The role of `templates.py` is confirmed and stated accurately in Section 3.3.
- The test counts (5, 9, 7, 7; total 28) match the screenshots.
- The generated reports in Figures 14–16 show the same incident identifier, status and fact lists.
- The next update time and contact route shown in the reports are the operator's entries.
- No screenshot contains personal or sensitive information.

Chapter 9 – Limitations and Future Enhancements

9.1 Current Limitations

- Incident details are provided through structured JSON; there is no automatic data collection.
- No live SIEM or SOAR integration is implemented.
- The operator must enter the next update time and the contact route; the system neither retrieves nor verifies them.
- The output validator checks structure and required indicators, not full semantic accuracy.
- Generated communication requires human review before distribution.
- The application supports three predefined audiences only.

9.2 Future Enhancements

The following are proposals for future work. **None of them is a current feature.**

- SIEM/SOAR integration to populate incident data from existing tooling.
- Configurable communication templates.
- Role-based approval workflows.
- Audit logging of generation and approval events.
- Additional stakeholder formats.
- Enhanced semantic consistency validation across reports.
- A web-based operator interface.
- Secure deployment and access controls.

Chapter 10 – Conclusion

Manually prepared incident updates are prone to inconsistency, to the wrong level of detail, to omissions and, most seriously, to presenting assumptions as facts or attributing responsibility without evidence. The Incident Communication Generator addresses these problems by capturing the incident once in a structured, schema-validated form and generating three audience-specific communications from it: a Technical Incident Update, a Management Incident Update and a Staff Security Notice.

Validation is applied at both ends of the pipeline. Input is checked against a JSON Schema before anything is generated, and each output is checked for required elements before it is saved. The operator must supply the next update time and approved contact route, and the communication rules prevent unsupported attribution and unvalidated recovery-time promises. The reports are saved as Markdown in designated directories.

The application was verified with an automated suite of 28 tests – 28 passed and none failed. These results show that the implementation behaves as specified; they do not claim that the system is production-ready, and the limitations in Chapter 9 remain. The project’s value lies in demonstrating that disciplined, consistent and responsible incident communication can be supported by a small, transparent and testable tool, while leaving judgement and accountability, through human review, with the people responsible for the incident.

Appendix A – Project Directory Structure

```
Incident_Communication_Generator/
|-- app/
|   |-- __init__.py
|   |-- main.py
|   |-- incident_parser.py
|   |-- communication_generator.py
|   |-- templates.py
|   `-- validator.py
|-- data/
|   |-- sample_incidents.json
|   `-- incident_schema.json
|-- tests/
|   |-- test_parser.py
|   |-- test_generator.py
|   |-- test_validator.py
|   `-- test_outputs.py
|-- outputs/
|   |-- technical_report/technical_update.md
|   |-- management_updates/management_update.md
|   `-- user_notifications/staff_notification.md
|-- evidence/
|   |-- Final_Test_Summary.png
|   |-- Generator_Test_Results.png
|   |-- Management_Update_Output.png
|   |-- Output_File_Test_Results.png
|   |-- Parser_Test_Results.png
|   |-- Staff_Notification_Output.png
|   |-- Technical_Update_Output.png
|   `-- Validator_Test_Results.png
|-- docs/
|   |-- architecture.md
|   `-- test_documentation.md
|-- incident_facts.json
|-- status_update_template.md
|-- requirements.txt
|-- README.md
`-- .gitignore
```

Appendix B – Important Execution Commands

Table 12 – Execution commands

Purpose	Command	Explanation
Create virtual environment	python -m venv .venv	Creates an isolated Python environment in .venv
Activate environment	.\.venv\Scripts\Activate.ps1	Activates it (after the process-scope execution-policy command in Section 4.1)
Install dependencies	pip install -r requirements.txt	Installs jsonschema 4.26.0 and other listed packages
Run application	python -m app.main	Runs the generator and prompts the operator
Run unit tests	python -m unittest discover -s tests -v	Discovers and runs all tests with verbose output
View output files	Get-Content outputs\technical_report\technical_update.md	Illustrative PowerShell command to display a generated report (confirm against the command you actually used); repeat for the other two files

Get-Content is a standard PowerShell command for displaying a file; any text editor may be used instead.

Appendix C – Evidence Index

Table 13 – Evidence index

#	Filename	Description	Report section
1	Parser_Test_Results.png	Parser unit test results	7.6.1
2	Generator_Test_Results.png	Generator unit test results	7.6.2
3	Validator_Test_Results.png	Validator unit test results	7.6.3
4	Output_File_Test_Results.png	Output file verification results	7.6.4
5	Technical_Update_Output.png	Generated technical update	5.1
6	Management_Update_Output.png	Generated management update	5.2
7	Staff_Notification_Output.png	Generated staff notification	5.3
8	Final_Test_Summary.png	Final test summary (28 passed, 0 failed)	7.5

Appendix D – Sample Incident Data

The listing below illustrates the **structure** of the incident record using the verified field names and the fictional scenario. It is not a verbatim copy of `incident_facts.json`; paste the actual file content here before submission if an exact listing is required. The value of `audiences` is shown as a placeholder because its exact format has not been verified.

```
{
  "incident_id": "RK-INC-TRAIN-204",
  "fictional_organisation": "Northstar Community Services",
  "status": "Investigating",
  "confirmed_facts": [
    "Repeated failed sign-in events were recorded for one account.",
    "The user denied two unexpected MFA prompts.",
    "A privilege event has an unclear change-ticket reference."
  ],
  "not_yet_confirmed": [
    "Whether credentials were stolen.",
    "Whether data was accessed or copied.",
    "Whether the privilege event was unauthorised."
  ],
  "audiences": [ ... ],
  "communication_rules": [
    "Separate confirmed facts from hypotheses.",
    "Do not name a person as responsible without evidence.",
    "Do not promise a recovery time until validated.",
    "Include the next update time and approved contact route."
  ]
}
```

Fields: `incident_id` identifies the incident; `fictional_organisation` names the fictional organisation; `status` records the current state; `confirmed_facts` and `not_yet_confirmed` hold the two evidence categories; `audiences` identifies the intended recipients; `communication_rules` records the constraints the reports must respect.

Appendix E – Test Summary

Table 14 – Test summary (verified results)

Test Module	Tests	Result
test_parser.py	5	Passed
test_generator.py	9	Passed
test_validator.py	7	Passed
test_outputs.py	7	Passed
Total	28	28 Passed, 0 Failed